МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

ЗВІТ

з лабораторної роботи №2
з дисципліни «Об’єктно-орієнтоване програмування»

Тема: Розробка графічного редактора об'єктів

Виконав:
студент 2 курсу Конкус Андрій Андрійович, групи ІМ-О51
Номер у списку: 4

Перевірив:
Рекечинський Дмитро Олександрович

Київ — 2026

1. МЕТА РОБОТИ ТА РОЗРАХУНОК ВАРІАНТУ

Мета: Отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів C++, запрограмувавши простий графічний редактор об’єктів в об'єктно-орієнтованому стилі у середовищі Win32.

Розрахунок варіанту для номера у журналі Ж = 4:

Параметр завдання	Формула (Ж = 4)	Результат розрахунку та вимоги
Структура збереження об'єктів	`Ж mod 3` (4 mod 3 = 1)	Статичний масив вказівників: `Shape *pcshape[104]` (N = 4 + 100).
"Гумовий" слід при вводі	`Ж mod 4` (4 mod 4 = 0)	Суцільна лінія чорного кольору.
Ввід прямокутника	`Ж mod 2` (4 mod 2 = 0)	Ввід по двом протилежним кутам.
Відображення прямокутника	`Ж mod 5` (4 mod 5 = 4)	Чорний контур прямокутника без заповнення.
Ввід еліпса	`Ж mod 2` (4 mod 2 = 0)	Ввід від центру до одного з кутів охоплюючого прямокутника.
Відображення еліпса	`Ж mod 5` (4 mod 5 = 4)	Чорний контур з кольоровим заповненням.
Колір заповнення еліпса	`Ж mod 6` (4 mod 6 = 4)	Рожевий / помаранчевий колір (`RGB(255, 192, 203)`).
Позначка поточного інструменту	`Ж mod 2` (4 mod 2 = 0)	Галочкою в меню (обробник `WM_INITMENUPOPUP`).

2. ІЄРАРХІЯ КЛАСІВ ТА ООП АРХІТЕКТУРА

Проєкт побудовано з використанням фундаментальних принципів ООП:

- **Інкапсуляція:** Координати поверхонь (`xs1, ys1, xs2, ys2`) приховані у захищеній секції (`protected`) базового класу `Shape`.
- **Поліморфізм:** Базовий клас `Shape` оголошує чисто віртуальний метод `virtual void Show(HDC hdc) = 0`, який перевизначається у класах-нащадках `RectShape` та `EllipseShape`.
- **Абстракція редакторів:** Для кожного об'єкта створено відповідний клас-редактор (`RectEditor`, `EllipseEditor`), який керує подіями миші (`WM_LBUTTONDOWN`, `WM_MOUSEMOVE`, `WM_LBUTTONUP`).

3. СХЕМА УСПАДКУВАННЯ КЛАСІВ (UML-ДІАГРАМА)

Нижче представлена UML-діаграма класів, що ілюструє ієрархію успадкування фігур та редакторів введення:

```
      +-------------------------------------------------------------------------------+ | S
```

4. ВИХІДНИЙ ТЕКСТ ПРОГРАМНИХ МОДУЛІВ

Файл LAB2.cpp (Головний модуль вікна та обробки меню):

```
#include <windows.h>
#include "resource.h"
#include "rect_editor.h"
#include "ellipse_editor.h"

static Shape* pcshape[104];
static int shapeCount = 0;

static RectEditor rectEditor;
static EllipseEditor ellipseEditor;
static bool isRectToolSelected = true;

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {
    case WM_COMMAND:
        switch (LOWORD(wParam))
        {
        case IDM_RECTANGLE: isRectToolSelected = true; break;
        case IDM_ELLIPSE:   isRectToolSelected = false; break;
        case IDM_EXIT:      DestroyWindow(hWnd); break;
        default: return DefWindowProcW(hWnd, message, wParam, lParam);
        }
        break;

    case WM_INITMENUPOPUP:
    {
        HMENU hMenu = (HMENU)wParam;
        CheckMenuItem(hMenu, IDM_RECTANGLE, isRectToolSelected ? MF_CHECKED :
MF_UNCHECKED);
        CheckMenuItem(hMenu, IDM_ELLIPSE, !isRectToolSelected ? MF_CHECKED :
MF_UNCHECKED);
    }
    break;

    case WM_LBUTTONDOWN:
        if (isRectToolSelected) rectEditor.OnLButtonDown(hWnd, lParam);
        else ellipseEditor.OnLButtonDown(hWnd, lParam);
        break;

    case WM_MOUSEMOVE:
        if (isRectToolSelected) rectEditor.OnMouseMove(hWnd, lParam);
        else ellipseEditor.OnMouseMove(hWnd, lParam);
        break;

    case WM_LBUTTONUP:
        if (isRectToolSelected) rectEditor.OnLButtonUp(hWnd, lParam, pcshape,
shapeCount);
        else ellipseEditor.OnLButtonUp(hWnd, lParam, pcshape, shapeCount);
        break;

    case WM_PAINT:
    {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        for (int i = 0; i < shapeCount; ++i) {
            if (pcshape[i]) pcshape[i]->Show(hdc);
        }
        EndPaint(hWnd, &ps);
    }
    break;

    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProcW(hWnd, message, wParam, lParam);
    }
    return 0;
```

```
WINAPI WinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPSTR lpCmdLine, _In_ int nCmdShow)
{
    const wchar_t CLASS_NAME[] = L"Lab2WindowClass";

    WNDCLASSW wc = {};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.hCursor = LoadCursor(NULL, IDC_CROSS);

    RegisterClassW(&wc);

    HMENU hMenu = LoadMenuW(hInstance, MAKEINTRESOURCEW(IDR_MYMENU));

    if (!hMenu)
    {
        HMENU hSubMenu = CreatePopupMenu();
        AppendMenuW(hSubMenu, MF_STRING, IDM_RECTANGLE, L"Rectangle");
        AppendMenuW(hSubMenu, MF_STRING, IDM_ELLIPSE, L"Ellipse");
        AppendMenuW(hSubMenu, MF_SEPARATOR, 0, NULL);
        AppendMenuW(hSubMenu, MF_STRING, IDM_EXIT, L"Exit");

        hMenu = CreateMenu();
        AppendMenuW(hMenu, MF_POPUP, (UINT_PTR)hSubMenu, L"Shapes");
    }

    HWND hWnd = CreateWindowExW(
        0, CLASS_NAME, L"Lab 2",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
        NULL, hMenu, hInstance, NULL
    );

    if (hWnd == NULL) return 0;

    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);

    MSG msg = {};
    while (GetMessage(&msg, NULL, 0, 0))
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return (int)msg.wParam;
}
```

Файл shape.h:

```
#ifndef SHAPE_H
#define SHAPE_H
#include <windows.h>

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
public:
    virtual ~Shape() {}
    void Set(long x1, long y1, long x2, long y2) {
        xs1 = x1; ys1 = y1; xs2 = x2; ys2 = y2;
    }
    virtual void Show(HDC hdc) = 0;
};
#endif
```

Файл rect_shape.h & rect_shape.cpp (Отображение прямоугольника без заливки):

```
// rect_shape.h
#ifndef RECT_SHAPE_H
#define RECT_SHAPE_H
#include "shape.h"

class RectShape : public Shape {
public:
    virtual void Show(HDC hdc) override;
};
#endif

// rect_shape.cpp
#include "rect_shape.h"

void RectShape::Show(HDC hdc) {
    HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    Rectangle(hdc, xs1, ys1, xs2, ys2);

    SelectObject(hdc, hOldPen);
    SelectObject(hdc, hOldBrush);
    DeleteObject(hPen);
}
```

Файл ellipse_shape.h & ellipse_shape.cpp (Розовая заливка):

```
// ellipse_shape.h
#ifndef ELLIPSE_SHAPE_H
#define ELLIPSE_SHAPE_H
#include "shape.h"

class EllipseShape : public Shape {
public:
    virtual void Show(HDC hdc) override;
};
#endif

// ellipse_shape.cpp
#include "ellipse_shape.h"

void EllipseShape::Show(HDC hdc) {
    HBRUSH hBrush = CreateSolidBrush(RGB(255, 192, 203)); // Рожевий колір
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    Ellipse(hdc, xs1, ys1, xs2, ys2);

    SelectObject(hdc, hOldPen);
    SelectObject(hdc, hOldBrush);
    DeleteObject(hPen);
    DeleteObject(hBrush);
}
```

Файл rect_editor.cpp (Ввод прямоугольника по 2 противоположным углам):

```
#include "rect_editor.h"
#include "rect_shape.h"

void RectEditor::OnLButtonDown(HWND hWnd, LPARAM lParam) {
    x1 = LOWORD(lParam); y1 = HIWORD(lParam);
    x2 = x1; y2 = y1;
    isDrawing = true;
    SetCapture(hWnd);
}

void RectEditor::OnMouseMove(HWND hWnd, LPARAM lParam) {
    if (!isDrawing) return;
    HDC hdc = GetDC(hWnd);
    SetROP2(hdc, R2_NOTXORPEN); // "Гумовий" слід
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    Rectangle(hdc, x1, y1, x2, y2); // Стирание предыдущей рамки
    x2 = LOWORD(lParam); y2 = HIWORD(lParam);
    Rectangle(hdc, x1, y1, x2, y2); // Отрисовка новой рамки

    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    ReleaseDC(hWnd, hdc);
}

void RectEditor::OnLButtonUp(HWND hWnd, LPARAM lParam, Shape** pcshape, int& count) {
    if (!isDrawing) return;
    isDrawing = false;
    ReleaseCapture();
    x2 = LOWORD(lParam); y2 = HIWORD(lParam);

    if (count < 104) {
        RectShape* pRect = new RectShape();
        pRect->Set(x1, y1, x2, y2);
        pcshape[count++] = pRect;
    }
    InvalidateRect(hWnd, NULL, TRUE);
}
```

Файл ellipse_editor.cpp (Ввод эллипса от центра к углу):

```
#include "ellipse_editor.h"
#include "ellipse_shape.h"
#include <cmath>

void EllipseEditor::OnLButtonUp(HWND hWnd, LPARAM lParam, Shape** pcshape, int& count)
{
    if (!isDrawing) return;
    isDrawing = false;
    ReleaseCapture();

    long cx = x1, cy = y1;
    long currX = LOWORD(lParam), currY = HIWORD(lParam);
    long dx = std::abs(currX - cx);
    long dy = std::abs(currY - cy);

    if (count < 104) {
        EllipseShape* pEllipse = new EllipseShape();
        pEllipse->Set(cx - dx, cy - dy, cx + dx, cy + dy);
        pcshape[count++] = pEllipse;
    }
    InvalidateRect(hWnd, NULL, TRUE);
}
```

5. ІЛЮСТРАЦІЇ РОБОТИ ПРОГРАМИ

Нижче наведено підтвердження роботи графічного редактора, створеного відповідно до розрахованого варіанту:

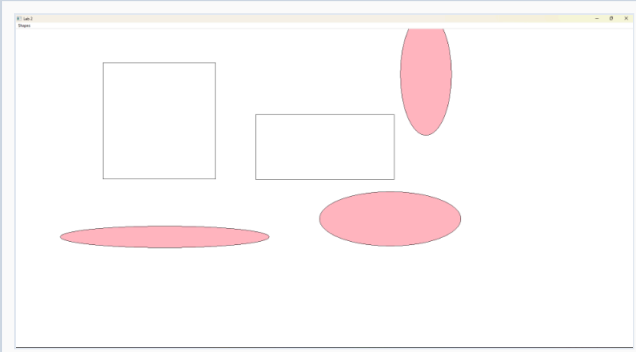


Рис. 1. Намальовані прямокутники (без заливки) та еліпси (з рожевою заливкою)

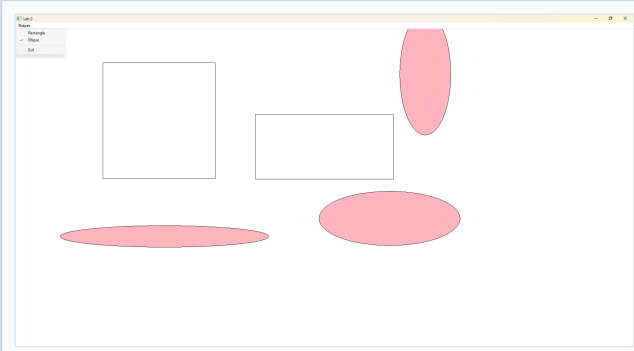


Рис. 2. Відображення галочки біля активного інструменту в меню Shapes

6. ВИСНОВКИ

Під час виконання лабораторної роботи № 2 було успішно реалізовано об'єктно-орієнтовану модель графічного редактора на базі Win32 API.

- Опановано використання базового абстрактного класу `Shape` із чисто віртуальною функцією `Show()` для забезпечення поліморфної поведінки об'єктів.
- Реалізовано класи-нащадки `RectShape` (прямокутники без заповнення) та `EllipseShape` (еліпси з рожевим заповненням) та статичний масив збереження `Shape* pcshape[104]`.
- Забезпечено алгоритми введення об'єктів за допомогою миші через відповідні класи-редактори `RectEditor` та `EllipseEditor` із промальовуванням "гумового" сліду.
- Налаштовано динамічне відмічання поточного інструменту галочкою в головному меню через обробку події `WM_INITMENUPOPUP`.